

# Lab07-ProducerConsumer

## 目标

采用多线程实现生产者消费者问题

## 工具

GCC编译器

代码块

```
1 sudo apt-get install build-essential
```

IDE

1. (推荐)Code::Block

代码块

```
1 sudo apt-get install codeblocks
```

## 相关函数

Posix 信号量Semaphore

- 创建与释放 :sem\_init()、sem\_destroy()
- 打开与关闭 :sem\_open()、sem\_close()、sem\_unlink()
- 使用 :
  - DOWN :sem\_wait()、sem\_trywait()
  - UP :sem\_post()
  - VALUE :sem\_getvalue()

具体声明:

代码块

```
1 #include <semaphore.h>
2
3 /*
4  * 初始化基于内存的信号量
```

```

5  * 参数:
6  *   sem      - 指向应用程序分配的sem_t变量
7  *   shared   - 0表示信号量在同一进程的线程间共享（随进程持续）;
8  *              非0表示信号量需放在共享内存区, 供多进程共享（随共享内存区持续）
9  *   value    - 信号量的初始值
10 * 返回值: 成功返回0, 出错返回-1
11 */
12 int sem_init(sem_t *sem, int shared, unsigned int value);
13
14 /*
15 * 摧毁指定的基于内存的信号量
16 * 参数:
17 *   sem - 指向要摧毁的信号量
18 * 返回值: 成功返回0, 出错返回-1
19 */
20 int sem_destroy(sem_t *sem);
21
22 /*
23 * 创建新的有名信号量或打开已存在的有名信号量
24 * 参数:
25 *   name     - IPC名字（可能对应文件系统中的路径名）
26 *   oflag    - 标志位, 可取值: 0、O_CREAT、O_CREAT|O_EXCL
27 *   mode     - 当oflag包含O_CREAT时, 需指定权限位（mode_t类型）
28 *   value    - 当oflag包含O_CREAT时, 需指定信号量初始值（unsigned int类型）
29 * 返回值: 成功返回指向信号量的指针, 出错返回SEM_FAILED
30 */
31 sem_t *sem_open(const char *name, int oflag, ...); // 原原型中name缺少指针符号*,
已修正
32
33 /*
34 * 关闭由sem_open打开的信号量
35 * 参数:
36 *   sem - 指向要关闭的信号量
37 * 返回值: 成功返回0, 出错返回-1
38 */
39 int sem_close(sem_t *sem);
40
41 /*
42 * 从系统中删除有名信号量
43 * 参数:
44 *   name - 要删除的信号量名字
45 * 返回值: 成功返回0, 出错返回-1
46 */
47 int sem_unlink(const char *name);
48
49 /*
50 * 等待信号量（P操作）

```

```

51  * 功能：若信号量值>0，则减1并立即返回；若值=0，则线程睡眠等待，
52  *      直到值>0后减1再返回
53  * 参数：
54  *      sem - 指向目标信号量
55  * 返回值：成功返回0，出错返回-1
56  */
57  int sem_wait(sem_t *sem);
58
59  /*
60  * 尝试等待信号量（非阻塞P操作）
61  * 功能：若信号量值>0，则减1并立即返回；若值=0，则返回EAGAIN错误
62  * 参数：
63  *      sem - 指向目标信号量
64  * 返回值：成功返回0，出错返回-1
65  */
66  int sem_trywait(sem_t *sem);
67
68  /*
69  * 发布信号量（V操作）
70  * 功能：将信号量值加1，唤醒等待该信号量值变为正数的任意线程
71  * 参数：
72  *      sem - 指向目标信号量
73  * 返回值：成功返回0，出错返回-1
74  */
75  int sem_post(sem_t *sem);
76
77  /*
78  * 获取信号量当前值
79  * 功能：通过valp返回信号量当前值；若信号量已上锁，返回0或负数（绝对值为等待线程数）
80  * 参数：
81  *      sem - 指向目标信号量
82  *      valp - 用于存储信号量当前值的指针
83  * 返回值：成功返回0，出错返回-1
84  */
85  int sem_getvalue(sem_t *sem, int *valp);

```

## Posix mutex互斥量

### 代码块

```

1  #include <pthread.h>
2  pthread_mutex_t mutex =PTHREAD_MUTEX_INITIALIZER;
3  int pthread_mutex_init(pthread_mutex_t *restrict mutex, const
    pthread_mutexattr_t *restrict attr);
4  int pthread_mutex_destroy(pthread_mutex_t *mutex);
5  int pthread_mutex_lock(pthread_mutex_t *mutex);

```

```
6  int pthread_mutex_trylock(pthread_mutex_t *mutex);
7  int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

实例:

代码块

```
1  pthread_mutex_t mutex;
2  void * thread_run(void *arg)
3  {
4      pthread_mutex_lock(&mutex);
5      //TODO
6      XXXXXXXXX
7      pthread_mutex_unlock(&mutex);
8      return 0;
9  }
10 int main(int argc, char *argv[])
11 {
12     pthread_t thread1, thread2;
13     pthread_mutex_init(&mutex, 0);
14     pthread_create(&thread1, NULL, thread_run, 0);
15     pthread_create(&thread2, NULL, thread_run, 0);
16     pthread_join(thread1, 0);
17     pthread_join(thread2, 0);
18     pthread_mutex_destroy(&mutex);
19     return 0;
20 }
```

随机函数

代码块

```
1  srand ((DWORD)time(NULL)); /* Seed the random # generator */
2  rand() /* generatea random*/
```

结构体声明

代码块

```
1  #define DATA_SIZE 256
2
3  // 产品结构体
4  typedef struct item_st {
5      int id;                // 产品编号
6      time_t timestamp_p;    // 生产时间
```

```

7      int produce_id;          // 生产者编号
8      DWORD checksum;         // 产品校验码
9      DWORD data[DATA_SIZE];  // 产品详细信息
10 } ITEM_ST;
11
12 // 缓冲区结构体
13 typedef struct buffer_st {
14     int id;                   // 缓冲区编号
15     bool isempty;             // 是否为空
16     ITEM_ST item;             // 具体产品
17     struct buffer_st* nextbuf; // 下一个缓冲区地址
18 } BUFFER_ST;

```

## 实验流程

1. 写一个c语言程序，实现I个生产者J个消费者问题，其中共享缓冲区的大小为N，所有生产者共生产K( $K > N$ )个产品后结束，所有消费者共消费K个产品后结束。具体要求：
  - 严格按时序输出每个生产者、消费者的行为，其中包括生产产品k、消费产品k、进入临界区、存入产品、取出产品、离开临界区；
  - 需要考虑边界（某生产者生产第K个产品后所有生产者结束；某消费者消费第K个产品后所有消费者结束）；
  - 需要考虑随机函数，生产者生产时需要一个随机时间；消费者消费时也需要一个随机时间；
  - 编号：无论生产者还是消费者都需要有编号；产品同样也需要编号；缓冲区的各个产品项也需要有编号；
  - 输出形式可以采用标准输出、图形动态显示及同时文字记录输出等方式，无论是生产者还是消费者，其主要输出内容如下： a) 进入临界区前，输出某某编号（生产者/消费者）线程准备进入临界区； b) 进入临界区后，输出某某编号（生产者/消费者）线程已进入临界区； c) 离开临界区后，输出某某编号（生产者/消费者）线程已离开临界区； d) 生产者生产一个产品时，需要输出产品信息； e) 生产者将产品放入缓冲区时，需要输出相关信息； f) 消费者将产品从缓冲区取出时，需要输出相关信息； g) 消费者消费一个产品时，需要输出产品信息；
  - 不能出现忙等待；
2. 撰写实验报告